

12 — File I/O

Daten dauerhaft speichern – Lesen und Schreiben mit Dateien

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Wozu Dateien?
2. `open()` und `close()`
3. Die Modi `r`, `w`, `a`
4. Schreiben mit `write()`
5. Lesen mit `read()` / `readline()` / Iteration
6. `with`-Block – automatisches Schließen
7. Pfade: absolut vs. relativ
8. Fehlerbehandlung beim File I/O

Lernziel: Daten über Programmstarts hinweg in Dateien sichern.

Wie wir arbeiten: Pro Konzept *Live-Demo* (ich) + Übung (Sie, NB 12).

Wozu Dateien?

Ohne Datei - Daten weg, sobald das Programm endet

```
warenkorb = ["Eco-Sneaker", "Hemp-High"]  
# Programm endet -> Liste ist verloren
```

Mit Datei - persistent

```
with open("warenkorb.txt", "w") as f:  
    f.write("Eco-Sneaker\nHemp-High\n")
```

Dateien sind das **Gedächtnis** Ihres Programms über Programmstarts hinweg.

open() und close()

```
f = open("preise.txt", "w")    # öffnen  
f.write("Eco-Sneaker 89.95\n") # arbeiten  
f.close()                     # schließen
```

- open() liefert ein **File-Objekt**
- close() schreibt den Puffer auf die Festplatte und gibt Ressourcen frei
- **Vergessen Sie's nicht** – ohne close() können Daten verloren gehen
- Besser: with-Block (gleich)

► **Live-Demo** - 01_datei_schreiben.py

✎ **Sofort ausprobieren** - Notebook 12, Angeleitete Übung 1.1: test.txt erstellen

Die Modi r, w, a

Modus	Bedeutung	Wenn Datei fehlt	Wenn Datei existiert
"r"	lesen	Fehler	wird gelesen
"w"	schreiben	wird angelegt	wird überschrieben
"a"	anhängen	wird angelegt	wird verlängert

```
open("log.txt", "w")    # neuer Tag, frisches Log
open("log.txt", "a")    # Einträge ans Ende anhängen
open("log.txt", "r")    # Log lesen
```

Vorsicht: "w" ist destruktiv – der vorherige Inhalt ist weg, ohne Warnung.

Schreiben mit write()

```
with open("tagesreport.txt", "w") as f:  
    f.write("Veggie Soles -- Tagesreport\n")  
    f.write("Eco-Sneaker 89.95\n")  
    f.write("Hemp-High 109.00\n")
```

- write(text) schreibt einen String – **keinen automatischen Zeilenumbruch**
- \n muss **manuell** gesetzt werden
- Mehrere write-Aufrufe hängen direkt aneinander

```
zeilen = ["Eco-Sneaker", "Hemp-High", "Bambus-Boot"]  
with open("modelle.txt", "w") as f:  
    f.write("\n".join(zeilen))
```

 **Sofort ausprobieren** - Notebook 12, Angeleitete Übung 2.1:
mein_text.txt mit Inhalt

Lesen - read() und readline()

```
with open("tagesreport.txt", "r") as f:
    inhalt = f.read()           # alles als ein String
print(inhalt)
```

Methode	Liefert
f.read()	gesamten Dateiinhalt als String
f.read(50)	maximal 50 Zeichen
f.readline()	nächste Zeile (mit \n)
f.readlines()	Liste aller Zeilen

Faustregel: kleine Dateien $\rightarrow$ `read()`. Große Dateien $\rightarrow$ Zeile für Zeile (gleich).

► **Live-Demo** - 02_datei_lesen.py

Zeile-für-Zeile mit for

```
with open("tagesreport.txt", "r") as f:
    for zeile in f:          # eine Zeile pro Durchlauf
        zeile = zeile.rstrip("\n")
        print(f"Eintrag: {zeile}")
```

- File-Objekte sind **iterierbar** – direkt mit for nutzbar
- Funktioniert auch für **riesige** Dateien (Datei wird streamend gelesen)
- `rstrip("\n")` entfernt das Zeilenumbruch-Zeichen

 **Sofort ausprobieren** - Notebook 12, Angeleitete Übung 3.1 / 3.2: Inhalt lesen, Zeilen zählen

Der with-Block - automatisches Schließen

```
with open("tagesreport.txt", "w") as f:  
    f.write("Hallo Welt\n")  
# hier ist die Datei garantiert geschlossen,  
# auch wenn im with-Block ein Fehler aufgetreten ist
```

- with ruft close() **automatisch** auf – auch im Fehlerfall
- Standard-Muster für **alle** File-Operationen
- Rückt den eigentlichen Code visuell ein – klar erkennbar

► **Live-Demo** – 03_with_block.py

Faustregel: in produktivem Code immer with. open + close ist nur der erste Lehr-Schritt.

Pfade: absolut vs. relativ

relativ -- ausgehend vom Arbeitsverzeichnis

```
open("daten/preise.txt")
```

absolut -- vollständiger Pfad ab Wurzel

```
open("/Users/anna/preise.txt")    # macOS / Linux
```

```
open(r"C:\Anna\preise.txt")       # Windows (Raw-String!)
```

Pfade - Sonderpfade & Regel

Sonderpfade	Bedeutung
.	aktuelles Verzeichnis
..	Verzeichnis darüber
daten/	Unterordner

Faustregel: in Lehr- und Kursprojekten **relative** Pfade. Die Datei landet dann da, wo Sie das Skript starten.

Fehlerbehandlung beim File I/O

```
try:
    with open("preise.txt", "r") as f:
        text = f.read()
except FileNotFoundError:
    print("Datei nicht gefunden -- starte mit leerem Bestand.")
    text = ""
except PermissionError:
    print("Keine Leseberechtigung.")
```

Exception	Wann?
FileNotFoundError	Datei (im r-Modus) existiert nicht
PermissionError	OS verweigert Zugriff
UnicodeDecodeError	Datei-Encoding passt nicht

► **Live-Demo** - 04_fehler_pfade.py

Cheat Card

Aufgabe	Syntax
Datei öffnen	<code>open(name, modus)</code>
Modus	"r" lesen, "w" schreiben, "a" anhängen
Sicher arbeiten	<code>with open(...) as f:</code>
Schreiben	<code>f.write("Text\n")</code>
Lesen alles	<code>f.read()</code>
Zeile für Zeile	<code>for zeile in f:</code>
Fehlerschutz	<code>try: ... except FileNotFoundError:</code>

Faustregel: with + relative Pfade + spezifische except-Blöcke. Damit decken Sie 90 Prozent aller Fälle ab.

Mit File I/O können Sie **echte** Daten verarbeiten – nicht mehr nur Konstanten im Code. Im nächsten Schritt:

- **Algorithmen** systematisch beschreiben
- **Pseudocode**, Struktogramme, Komplexität
- Vom Problem über den Plan zum Code

Aus Datei lesen → Algorithmus anwenden → Ergebnis schreiben – das ist der Standard-Bauplan datenverarbeitender Programme.

Heute geübt

- ✓ `open()` / `close()` – Datei öffnen und schließen
- ✓ Modi `r`, `w`, `a` – lesen, überschreiben, anhängen
- ✓ `write()` – Strings in eine Datei schreiben
- ✓ `read()` / Zeile-für-Zeile mit `for`
- ✓ `with`-Block – garantiertes Schließen
- ✓ Absolute vs. relative Pfade
- ✓ `try / except FileNotFoundError` – robust gegen fehlende Dateien

Zur Vertiefung im Notebook 12:

- Angeleitete Übungen 1.1, 1.2, 2.1, 2.2, 3.1, 3.2, 4.1, 4.2 (sind Sie schon mitgegangen)
- Abschnitt zum `with`-Statement – Notebook 12, Kap. “Sicheres Arbeiten”
- Abschlussübungen am Notebook-Ende mit Musterlösungen